

DSA Practical No: 03

Name: More Sahil Sandip

PRN: B26ET2022

Div: SY3

Class: C3

INPUT

```
#include <stdio.h>

void display(int arr[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");
}

/* Bubble Sort */
void bubbleSort(int arr[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
```

```

    {
        temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}
}

```

/ Selection Sort */*

```
void selectionSort(int arr[], int n)
```

```

{
    int i, j, min, temp;

    for (i = 0; i < n - 1; i++)
    {
        min = i;

        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }
    }

```

```
    temp = arr[i];
```

```
    arr[i] = arr[min];
```

```
    arr[min] = temp;
}
}
```

/ Insertion Sort */*

```
void insertionSort(int arr[], int n)
```

```
{
    int i, j, key;
```

```
    for (i = 1; i < n; i++)
```

```
    {
        key = arr[i];
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key)
```

```
        {
            arr[j + 1] = arr[j];
            j--;
        }
```

```
        arr[j + 1] = key;
```

```
    }
}
```

/ Partition function for Quick Sort */*

```
int partition(int arr[], int low, int high)
```

```
{
    int pivot, i, j, temp;
```

```
pivot = arr[high];
```

```
i = low - 1;
```

```
for (j = low; j < high; j++)
```

```
{
```

```
    if (arr[j] < pivot)
```

```
    {
```

```
        i++;
```

```
        temp = arr[i];
```

```
        arr[i] = arr[j];
```

```
        arr[j] = temp;
```

```
    }
```

```
}
```

```
temp = arr[i + 1];
```

```
arr[i + 1] = arr[high];
```

```
arr[high] = temp;
```

```
return i + 1;
```

```
}
```

```
/* Quick Sort */
```

```
void quickSort(int arr[], int low, int high)
```

```
{
```

```
    int pi;
```

```

    if (low < high)
    {
        pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

/* Merge function */
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];
    int i, j, k;

    i = low;
    j = mid + 1;
    k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {

```

```
    temp[k] = arr[j];  
    j++;  
}
```

```
    k++;  
}
```

```
while (i <= mid)  
{  
    temp[k] = arr[i];  
    i++;  
    k++;  
}
```

```
while (j <= high)  
{  
    temp[k] = arr[j];  
    j++;  
    k++;  
}
```

```
for (i = low; i <= high; i++)  
{  
    arr[i] = temp[i];  
}  
}
```

```
/* Merge Sort */
```

```
void mergeSort(int arr[], int low, int high)
```

```
{
```

```
    int mid;
```

```
    if (low < high)
```

```
    {
```

```
        mid = (low + high) / 2;
```

```
        mergeSort(arr, low, mid);
```

```
        mergeSort(arr, mid + 1, high);
```

```
        merge(arr, low, mid, high);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int arr[100];
```

```
    int n, i, choice;
```

```
    char ch;
```

```
    do
```

```
    {
```

```
        printf(" MAIN MENU\n");
```

```
        printf("1. Bubble Sort\n");
```

```
printf("2. Selection Sort\n");
```

```
printf("3. Insertion Sort\n");
```

```
printf("4. Quick Sort\n");
```

```
printf("5. Merge Sort\n");
```

```
printf("\nEnter your choice: ");
```

```
scanf("%d", &choice);
```

```
printf("Enter number of elements: ");
```

```
scanf("%d", &n);
```

```
printf("Enter %d unsorted elements:\n", n);
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    scanf("%d", &arr[i]);
```

```
}
```

```
printf("\nOriginal array: ");
```

```
display(arr, n);
```

```
switch (choice)
```

```
{
```

```
    case 1:
```

```
        bubbleSort(arr, n);
```

```
        printf("Sorted using Bubble Sort: ");
```

```
        display(arr, n);
```

```
        break;
```

case 2:

```
selectionSort(arr, n);  
printf("Sorted using Selection Sort: ");  
display(arr, n);  
break;
```

case 3:

```
insertionSort(arr, n);  
printf("Sorted using Insertion Sort: ");  
display(arr, n);  
break;
```

case 4:

```
quickSort(arr, 0, n - 1);  
printf("Sorted using Quick Sort: ");  
display(arr, n);  
break;
```

case 5:

```
mergeSort(arr, 0, n - 1);  
printf("Sorted using Merge Sort: ");  
display(arr, n);  
break;
```

default:

```
printf("Invalid choice!\n");
```

```
}
```

```
    printf("\nDo you want to continue? (y/n): ");  
    scanf(" %c", &ch);  
  
} while (ch == 'y' || ch == 'Y');  
  
printf("\nProgram ended.\n");  
  
return 0;  
}
```

OUTPUT :

```
● PS D:\dsa\Experiment-03-Sorting> .\program.exe  
MAIN MENU  
1. Bubble Sort  
2. Selection Sort  
3. Insertion Sort  
4. Quick Sort  
5. Merge Sort  
  
Enter your choice: 1  
Enter number of elements: 3  
Enter 3 unsorted elements:  
34  
55  
2  
  
Original array: 34 55 2  
Sorted using Bubble Sort: 2 34 55
```

Do you want to continue? (y/n): y

MAIN MENU

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 2

Enter number of elements: 3

Enter 3 unsorted elements:

3

2

1

Original array: 3 2 1

Sorted using Selection Sort: 1 2 3

Do you want to continue? (y/n): y

MAIN MENU

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 3

Enter number of elements: 4

Enter 4 unsorted elements:

6

8

7

9

Original array: 6 8 7 9

Sorted using Insertion Sort: 6 7 8 9

Do you want to continue? (y/n): y

MAIN MENU

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 4

Enter number of elements: 4

Enter 4 unsorted elements:

1

1

6

7

Original array: 1 1 6 7

Sorted using Quick Sort: 1 1 6 7

Do you want to continue? (y/n): y

MAIN MENU

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 5

Enter number of elements: 5

Enter 5 unsorted elements:

80

70

90

50

40

Original array: 80 70 90 50 40

Sorted using Merge Sort: 40 50 70 80 90